

RISING WATERS

Project Description:

The Flood Prediction System is a Machine Learning-based web application developed to predict the likelihood of flooding using environmental and weather-related parameters. The system utilizes a trained XGBoost model to analyse user-provided input data and generate accurate flood predictions.

The application is built using Python and Flask, providing a simple and interactive web interface where users can enter the required parameters and instantly receive prediction results. The project demonstrates the integration of machine learning with web technologies to support early flood risk assessment and improve disaster preparedness.

The system focuses on providing fast, reliable, and user-friendly predictions while showcasing the practical application of data preprocessing, model training, and web deployment.

Scenarios

Scenario 1:

A disaster management officer wants to assess the possibility of flooding in a region based on current environmental conditions. The system predicts whether flood conditions are likely, enabling timely preventive measures.

Scenario 2:

Researchers use the application to evaluate different environmental parameter combinations and analyze their impact on flood prediction.

Scenario 3:

Educational institutions use the application as a demonstration of how Machine Learning models can be integrated with Flask to develop real-world predictive systems.

Technical Architecture

Description : The system follows a layered architecture consisting of the Presentation Layer, Application Layer, Machine Learning Layer, Data Processing Layer, and Data Layer.

The Presentation Layer provides a user-friendly interface built using HTML and CSS where users enter flood-related parameters.

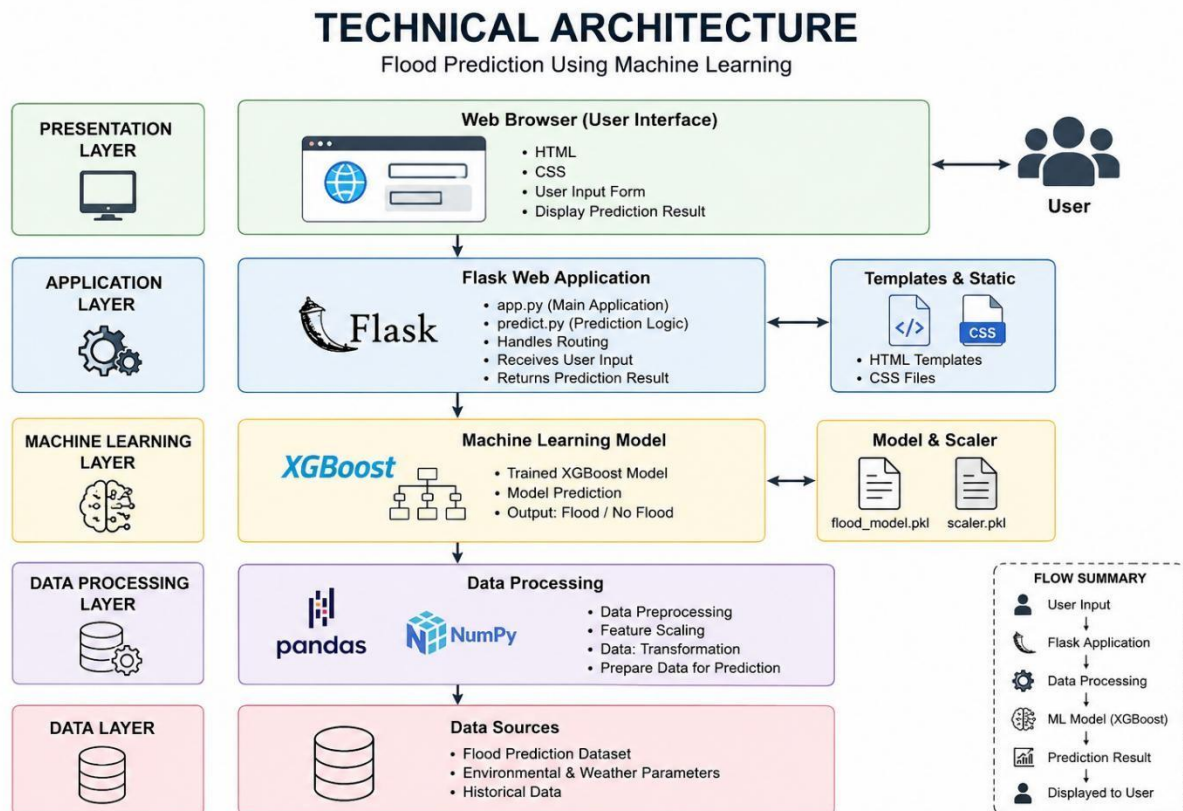
The Application Layer is implemented using the Flask framework. It receives user input, validates the data, loads the trained model, and returns the prediction result.

The Machine Learning Layer contains the trained XGBoost model, which processes the input features and predicts the flood risk.

The Data Processing Layer uses Pandas and NumPy to prepare and scale input data before passing it to the prediction model.

The Data Layer stores the flood prediction dataset, trained machine learning model, and scaler required for prediction.

Visual Representation



Pre-requisites:

Hardware Requirements:

- Laptop/Desktop with Windows 10/11, Linux, or macOS
- Minimum 4 GB RAM (8 GB Recommended)
- Minimum 2 GB Free Disk Space
- Stable Internet Connection (for downloading libraries)

Software Requirements:

- Python 3.10 or above
- Visual Studio Code
- Git
- GitHub Account
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)

Python Libraries:

- Flask
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Joblib

Development Tools:

- VS Code Terminal / Command Prompt
- pip (Python Package Manager)

Required Project Files:

- app.py
- train_model.py
- predict.py
- requirements.txt
- templates/index.html
- static/style.css
- model/flood_model.pkl
- model/scaler.pkl
- Dataset (CSV file)

Project Workflow

Milestone 1: Problem Analysis and Dataset Collection

Activity 1.1: Identified the problem of predicting flood risk using environmental and rainfall parameters.

Activity 1.2: Collected a flood prediction dataset containing rainfall, humidity, temperature, river level, and other related features.

Activity 1.3: Performed data cleaning, preprocessing, and exploratory data analysis to understand feature relationships.

Milestone 2: Machine Learning Model Development

Activity 2.1: Split the dataset into training and testing datasets.

Activity 2.2: Trained the XGBoost Machine Learning model for flood prediction.

Activity 2.3: Evaluated the model using accuracy, precision, recall, and F1-score.

Activity 2.4: Saved the trained model using Pickle for deployment.

Milestone 3: Backend Development

Activity 3.1: Developed the backend using Flask.

Activity 3.2: Created routes for user input, prediction, and result display.

Activity 3.3: Loaded the trained model dynamically for making predictions.

Milestone 4: Frontend Development

Activity 4.1: Designed an interactive user interface using HTML and CSS.

Activity 4.2: Developed an input form for environmental parameters.

Activity 4.3: Displayed flood prediction results in a simple and user-friendly format

Milestone 5: Testing and Deployment

Activity 5.1: Tested the application using different input values.

Activity 5.2: Verified prediction accuracy and application performance. **Activity 5.3:** Fixed bugs and improved code readability and usability

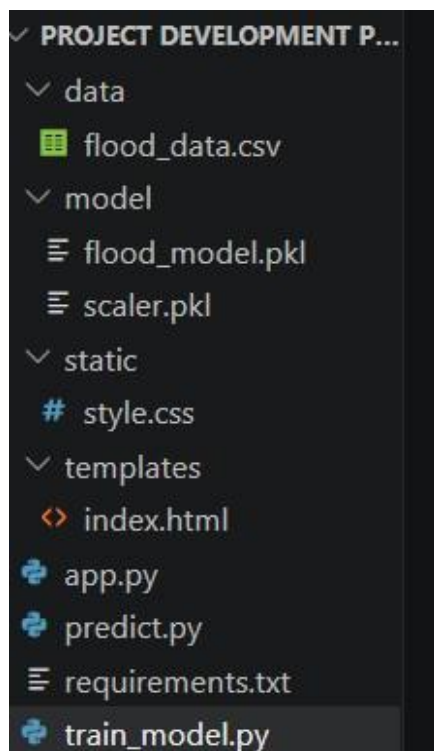
Milestone 6: Documentation and Conclusion

Prepared project documentation including architecture, implementation details, testing results, screenshots, and future enhancements.

Milestone 1: Project Setup & Dataset Preparation

This milestone focused on setting up the project environment and preparing the dataset required for flood prediction. The project structure was created in Visual Studio Code, necessary Python libraries were installed, and the flood prediction dataset was imported for preprocessing. The dataset was analysed to identify relevant features required for training the machine learning model

Screenshot-1 Project structure



Screenshot-2 Requirements for the project

* Running on http://127.0.0.1:5000
Press CTRL+C to quit
On the right, a 'Set BYOK' utility models panel is visible with a 'Configure' button." data-bbox="122 121 898 501"/>

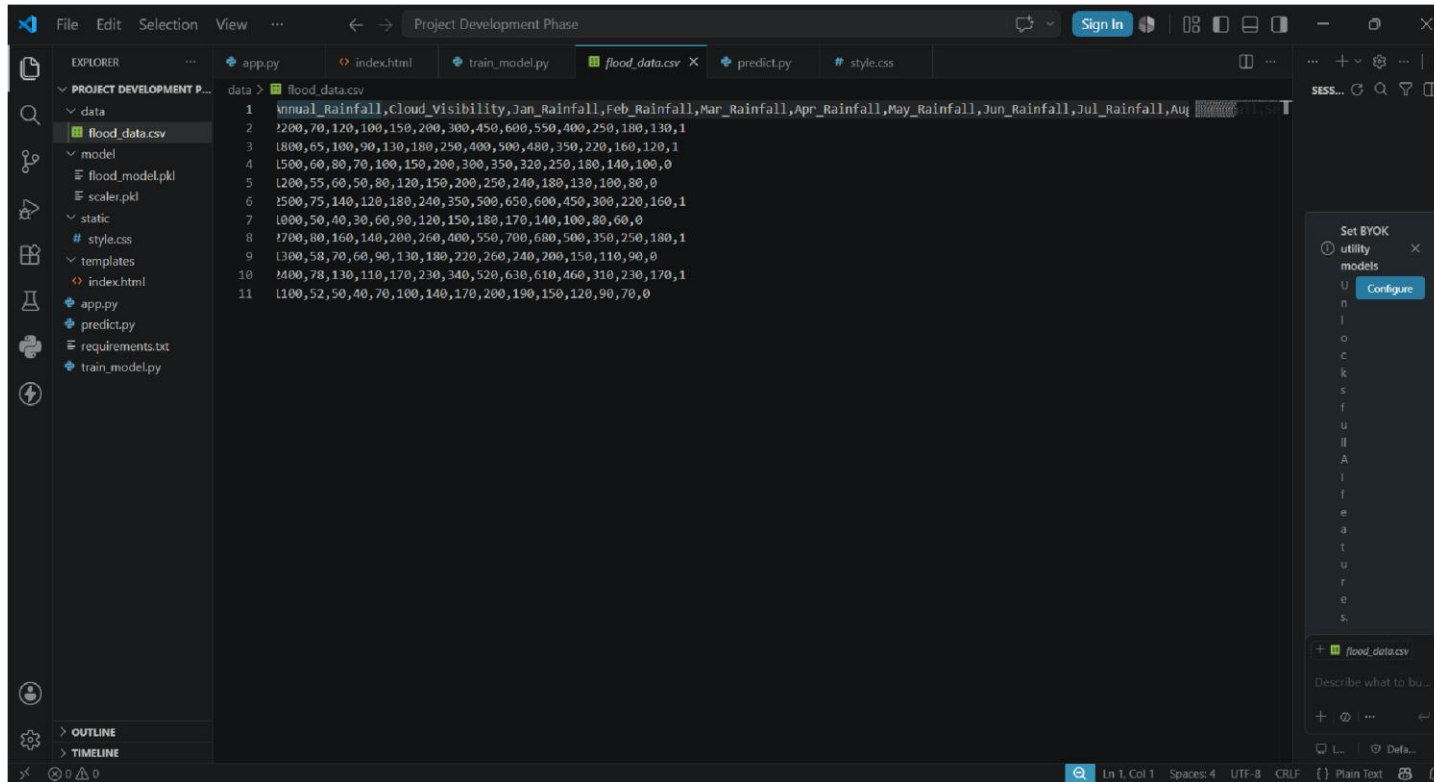
```
1 flask==1.0.3
2 pandas==2.2.2
3 numpy==1.26.4
4 scikit-learn==1.5.1
5 joblib==1.4.2
6 xgboost==2.1.1
```

PS D:\Project Development Phase> & C:\Users\144vi\AppData\Local\Microsoft\WindowsApps\python3.11.exe "d:/Project Development Phase/app.py"

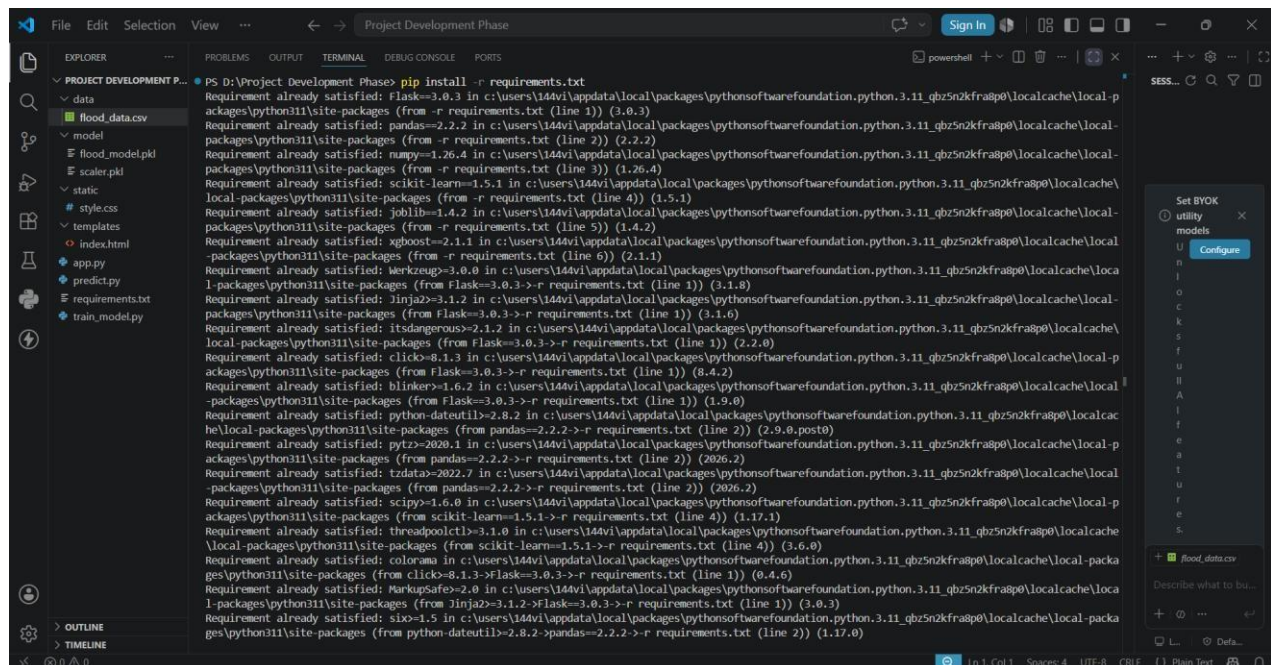
* Running on http://127.0.0.1:5000

Press CTRL+C to quit

Screenshot-3 Dataset for the model



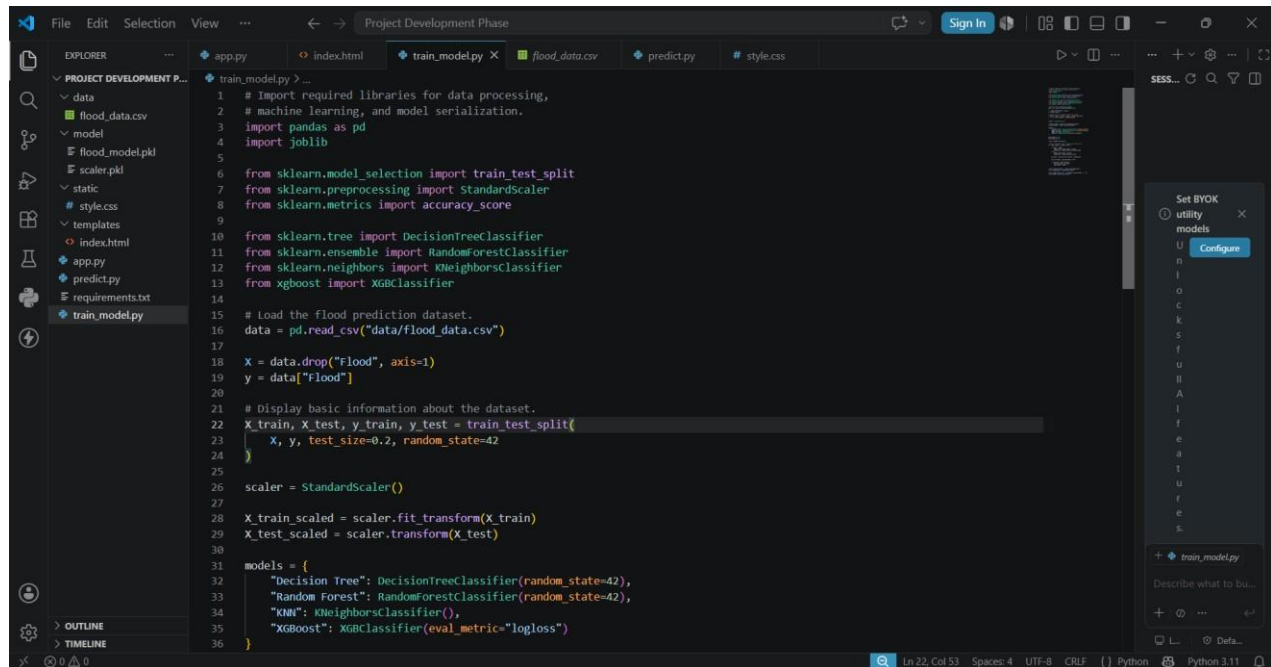
Screenshot-3 Terminal showing installed python libraries for the project



Milestone 2: Data Preprocessing & Model Training

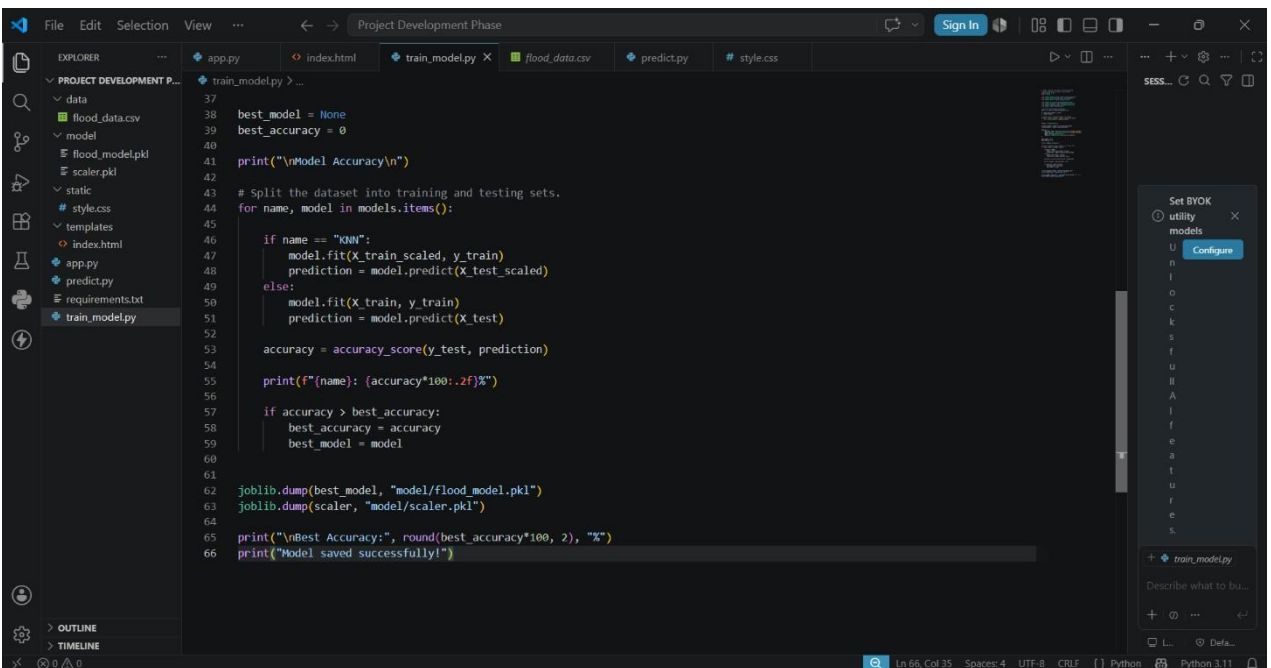
During this milestone, the dataset was pre-processed by selecting the required features and preparing the data for training. The dataset was split into training and testing sets. A machine learning model was trained using the processed dataset and saved for future predictions.

Screenshot-1,2 code to train the model



```

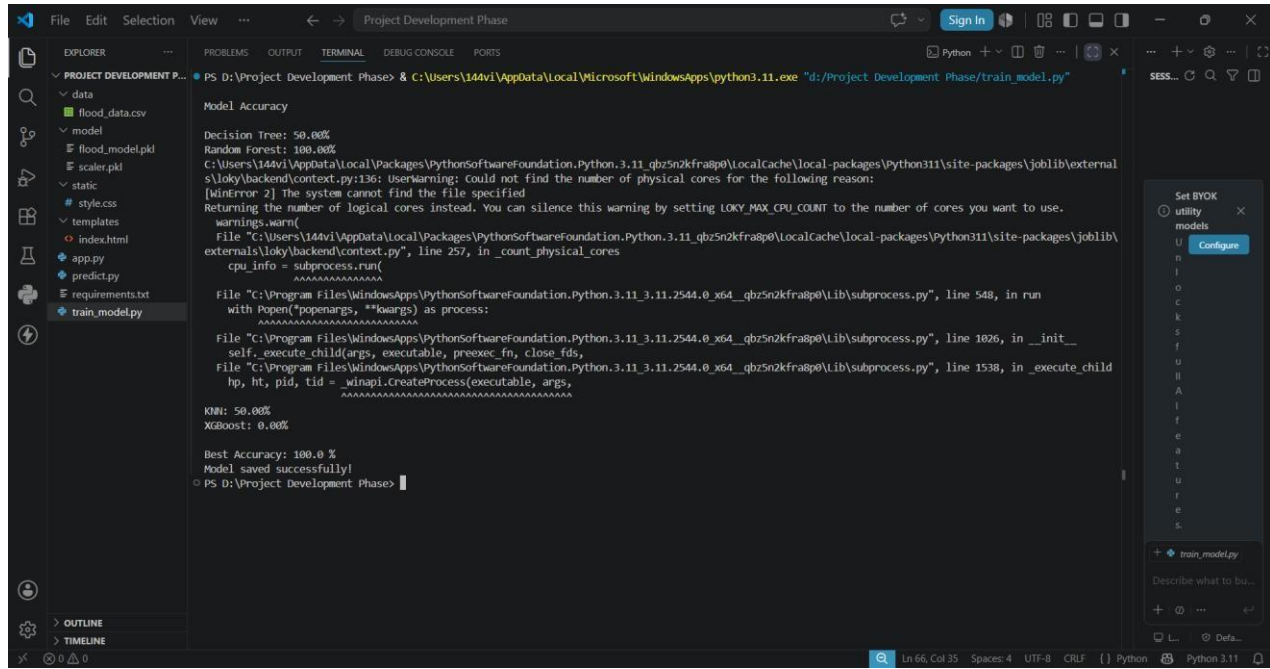
1  # Import required libraries for data processing,
2  # machine learning, and model serialization.
3  import pandas as pd
4  import joblib
5
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8  from sklearn.metrics import accuracy_score
9
10 from sklearn.tree import DecisionTreeClassifier
11 from sklearn.ensemble import RandomForestClassifier
12 from sklearn.neighbors import KNeighborsClassifier
13 from xgboost import XGBClassifier
14
15 # Load the flood prediction dataset.
16 data = pd.read_csv("data/flood_data.csv")
17
18 X = data.drop("Flood", axis=1)
19 y = data["Flood"]
20
21 # Display basic information about the dataset.
22 X_train, X_test, y_train, y_test = train_test_split(
23     X, y, test_size=0.2, random_state=42
24 )
25
26 scaler = StandardScaler()
27
28 X_train_scaled = scaler.fit_transform(X_train)
29 X_test_scaled = scaler.transform(X_test)
30
31 models = {
32     "Decision Tree": DecisionTreeClassifier(random_state=42),
33     "Random Forest": RandomForestClassifier(random_state=42),
34     "KNN": KNeighborsClassifier(),
35     "XGBoost": XGBClassifier(eval_metric="logloss")
36 }
  
```



```

37 best_model = None
38 best_accuracy = 0
39
40 print("\nModel Accuracy\n")
41
42 # Split the dataset into training and testing sets.
43 for name, model in models.items():
44
45     if name == "KNN":
46         model.fit(X_train_scaled, y_train)
47         prediction = model.predict(X_test_scaled)
48     else:
49         model.fit(X_train, y_train)
50         prediction = model.predict(X_test)
51
52     accuracy = accuracy_score(y_test, prediction)
53
54     print(f"{name}: {accuracy*100:.2f}%")
55
56     if accuracy > best_accuracy:
57         best_accuracy = accuracy
58         best_model = model
59
60
61 joblib.dump(best_model, "model/flood_model.pkl")
62 joblib.dump(scaler, "model/scaler.pkl")
63
64 print("\nBest Accuracy:", round(best_accuracy*100, 2), "%")
65 print("Model saved successfully!")
  
```


Screenshot -3 Terminal after running the training code



```

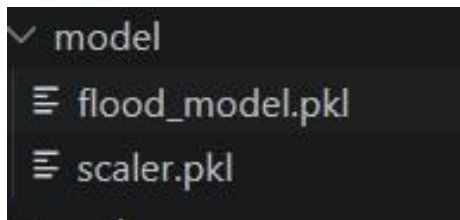
PS D:\Project Development Phase> & C:\Users\144vi\AppData\Local\Microsoft\WindowsApps\python3.11.exe "d:/Project Development Phase/train_model.py"

Model Accuracy
Decision Tree: 50.00%
Random Forest: 100.00%
C:\Users\144vi\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\joblib\external
s\loky\backend\context.py:136: UserWarning: could not find the number of physical cores for the following reason:
[Warning 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
  warnings.warn(
File "C:\Users\144vi\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\joblib\
externals\loky\backend\context.py", line 257, in _count_physical_cores
  cpu_info = subprocess.run(
File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.11_3.11.2544.0_x64_qbz5n2kfra8p0\Lib\subprocess.py", line 548, in run
  with Popen(*popenargs, **kwargs) as process:
File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.11_3.11.2544.0_x64_qbz5n2kfra8p0\Lib\subprocess.py", line 1026, in __init__
  self._execute_child(args, executable, preexec_fn, close_fds,
File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.11_3.11.2544.0_x64_qbz5n2kfra8p0\Lib\subprocess.py", line 1538, in _execute_child
  hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
  ~~~~~

KNN: 50.00%
XGBoost: 0.00%

Best Accuracy: 100.0 %
Model saved successfully!
PS D:\Project Development Phase>
  
```

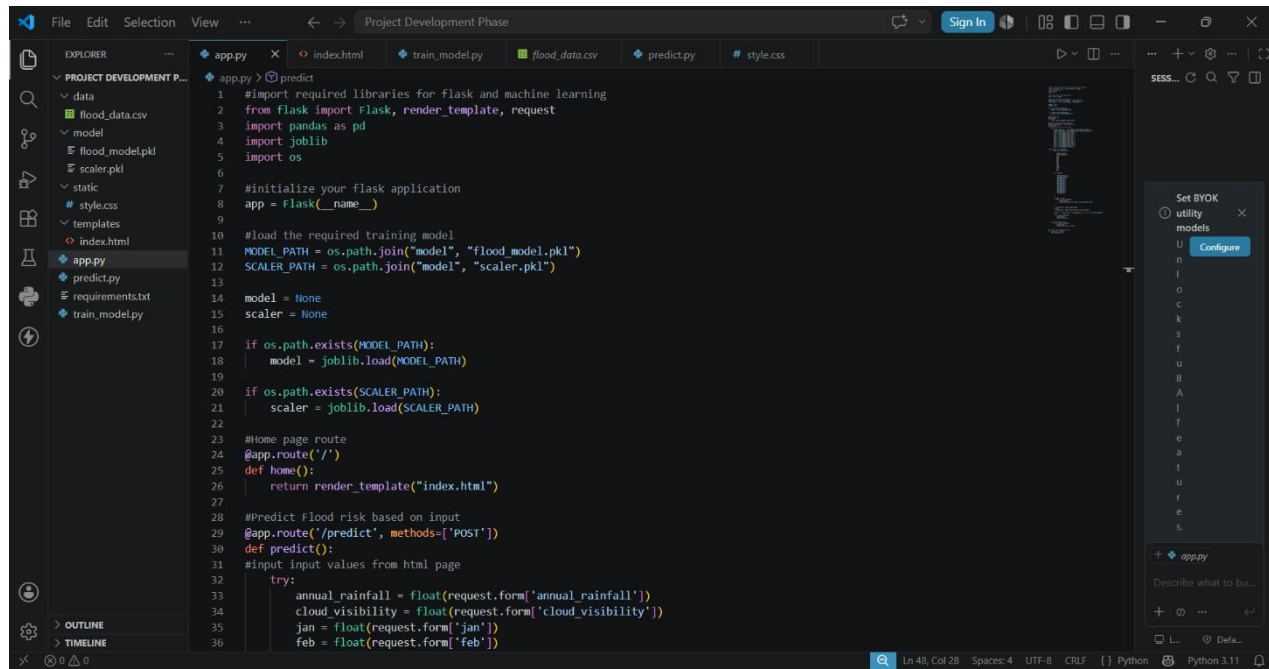
Screenshot-4 Model.pkl files created in model folder



Milestone 3: Flask Web Application Development

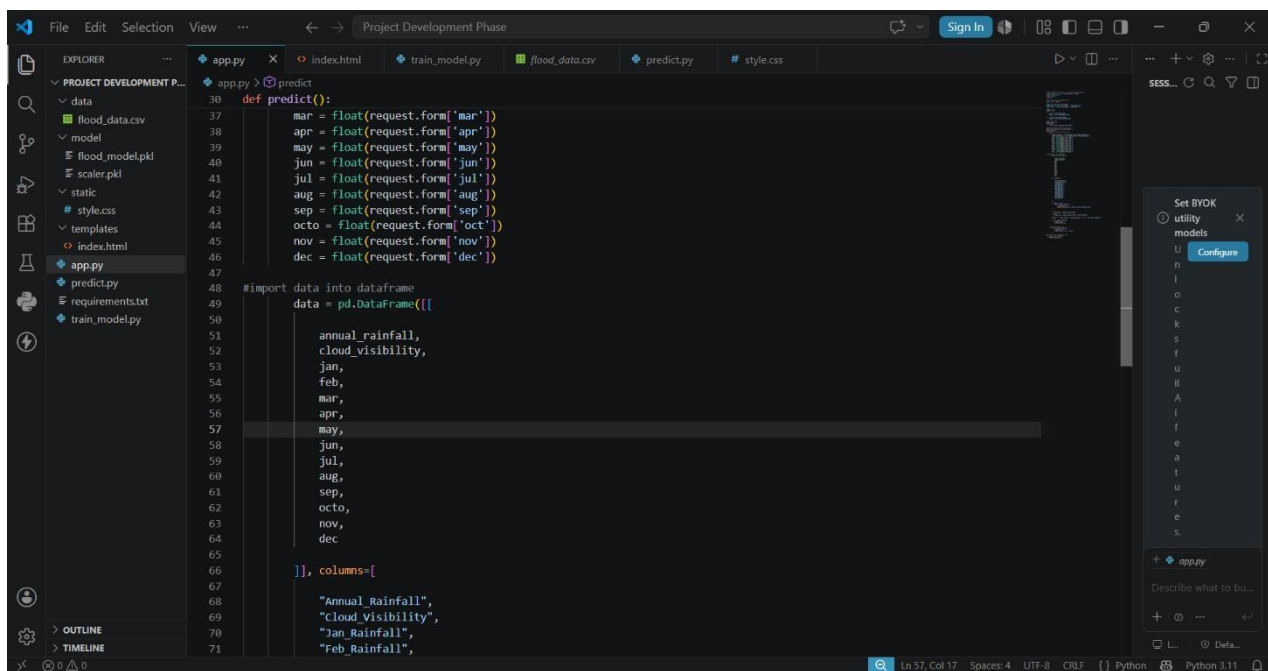
The trained machine learning model was integrated with a Flask web application. User input forms were created to collect flood-related parameters. Backend routes were implemented to receive user input, load the trained model, and generate prediction results.

Screenshot -1,2,3 showing the code used for app development



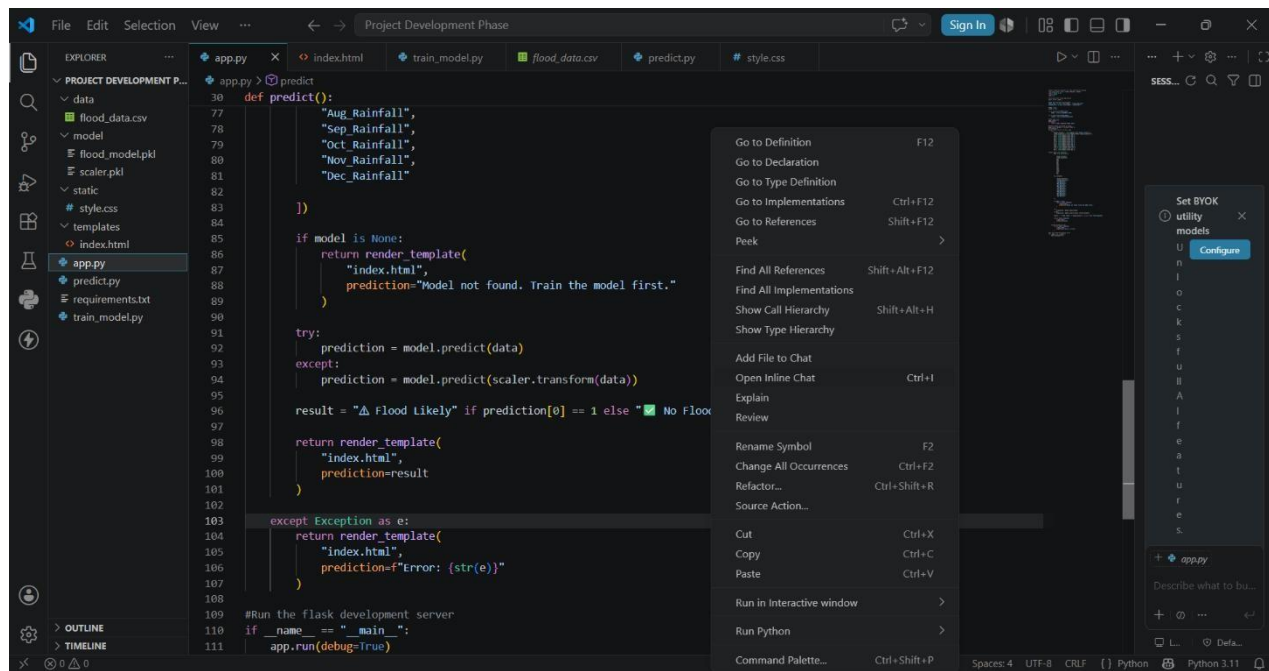
```

1  #import required libraries for flask and machine learning
2  from flask import Flask, render_template, request
3  import pandas as pd
4  import joblib
5  import os
6
7  #initialize your flask application
8  app = Flask(__name__)
9
10 #load the required training model
11 MODEL_PATH = os.path.join("model", "flood_model.pkl")
12 SCALER_PATH = os.path.join("model", "scaler.pkl")
13
14 model = None
15 scaler = None
16
17 if os.path.exists(MODEL_PATH):
18     model = joblib.load(MODEL_PATH)
19
20 if os.path.exists(SCALER_PATH):
21     scaler = joblib.load(SCALER_PATH)
22
23 #Home page route
24 @app.route('/')
25 def home():
26     return render_template("index.html")
27
28 #Predict Flood risk based on input
29 @app.route('/predict', methods=['POST'])
30 def predict():
31     #input input values from html page
32     try:
33         annual_rainfall = float(request.form['annual_rainfall'])
34         cloud_visibility = float(request.form['cloud_visibility'])
35         jan = float(request.form['jan'])
36         feb = float(request.form['feb'])
  
```



```

37     mar = float(request.form['mar'])
38     apr = float(request.form['apr'])
39     may = float(request.form['may'])
40     jun = float(request.form['jun'])
41     jul = float(request.form['jul'])
42     aug = float(request.form['aug'])
43     sep = float(request.form['sep'])
44     octo = float(request.form['oct'])
45     nov = float(request.form['nov'])
46     dec = float(request.form['dec'])
47
48 #import data into dataframe
49 data = pd.DataFrame([
50
51     annual_rainfall,
52     cloud_visibility,
53     jan,
54     feb,
55     mar,
56     apr,
57     may,
58     jun,
59     jul,
60     aug,
61     sep,
62     octo,
63     nov,
64     dec
65
66 ], columns=[
67
68     "Annual_Rainfall",
69     "Cloud_Visibility",
70     "Jan_Rainfall",
71     "Feb_Rainfall",
  
```



```

def predict():
    "Aug_Rainfall",
    "Sep_Rainfall",
    "Oct_Rainfall",
    "Nov_Rainfall",
    "Dec_Rainfall"

    if model is None:
        return render_template(
            "index.html",
            prediction="Model not found. Train the model first."
        )

    try:
        prediction = model.predict(data)
    except:
        prediction = model.predict(scaler.transform(data))

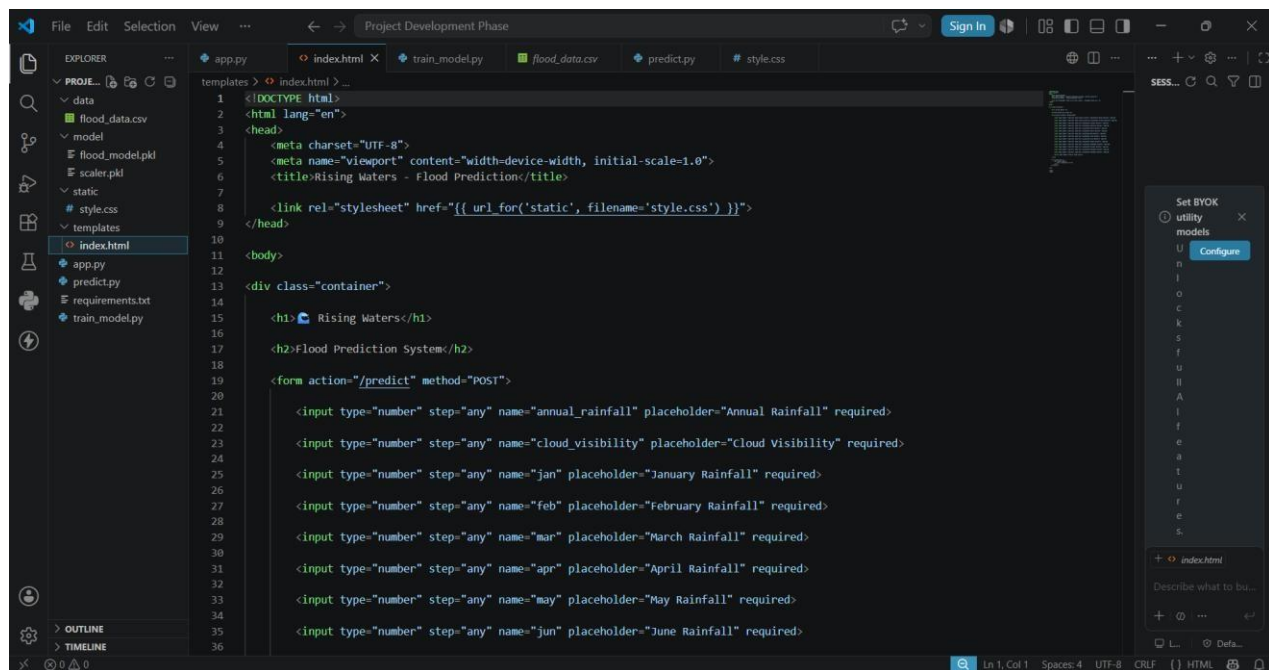
    result = "▲ Flood Likely" if prediction[0] == 1 else "■ No Flood"

    return render_template(
        "index.html",
        prediction=result
    )

except Exception as e:
    return render_template(
        "index.html",
        prediction=f"Error: {str(e)}"
    )

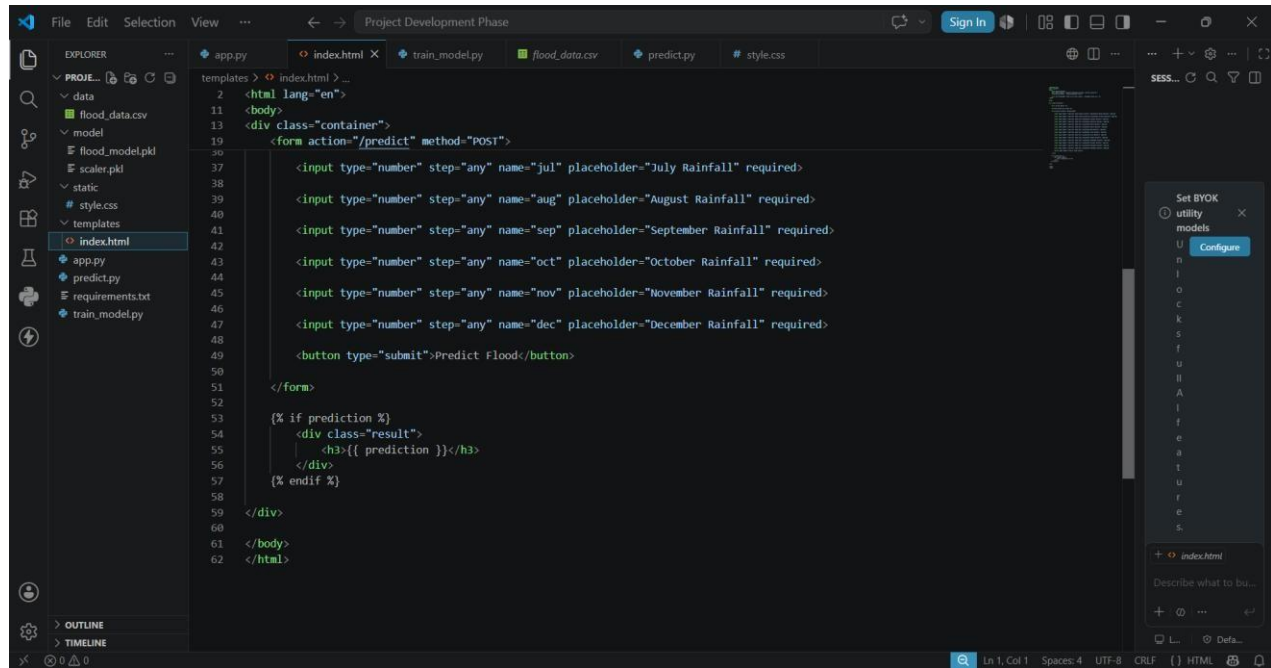
#Run the flask development server
if __name__ == "__main__":
    app.run(debug=True)
  
```

Screenshot-4,5 Html code used in the project



```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Rising Waters - Flood Prediction</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <div class="container">
    <h1> Rising Waters</h1>
    <h2>Flood Prediction System</h2>
    <form action="/predict" method="POST">
      <input type="number" step="any" name="annual_rainfall" placeholder="Annual Rainfall" required>
      <input type="number" step="any" name="cloud_visibility" placeholder="Cloud Visibility" required>
      <input type="number" step="any" name="jan" placeholder="January Rainfall" required>
      <input type="number" step="any" name="feb" placeholder="February Rainfall" required>
      <input type="number" step="any" name="mar" placeholder="March Rainfall" required>
      <input type="number" step="any" name="apr" placeholder="April Rainfall" required>
      <input type="number" step="any" name="may" placeholder="May Rainfall" required>
      <input type="number" step="any" name="jun" placeholder="June Rainfall" required>
    </form>
  </div>
</body>
</html>
  
```

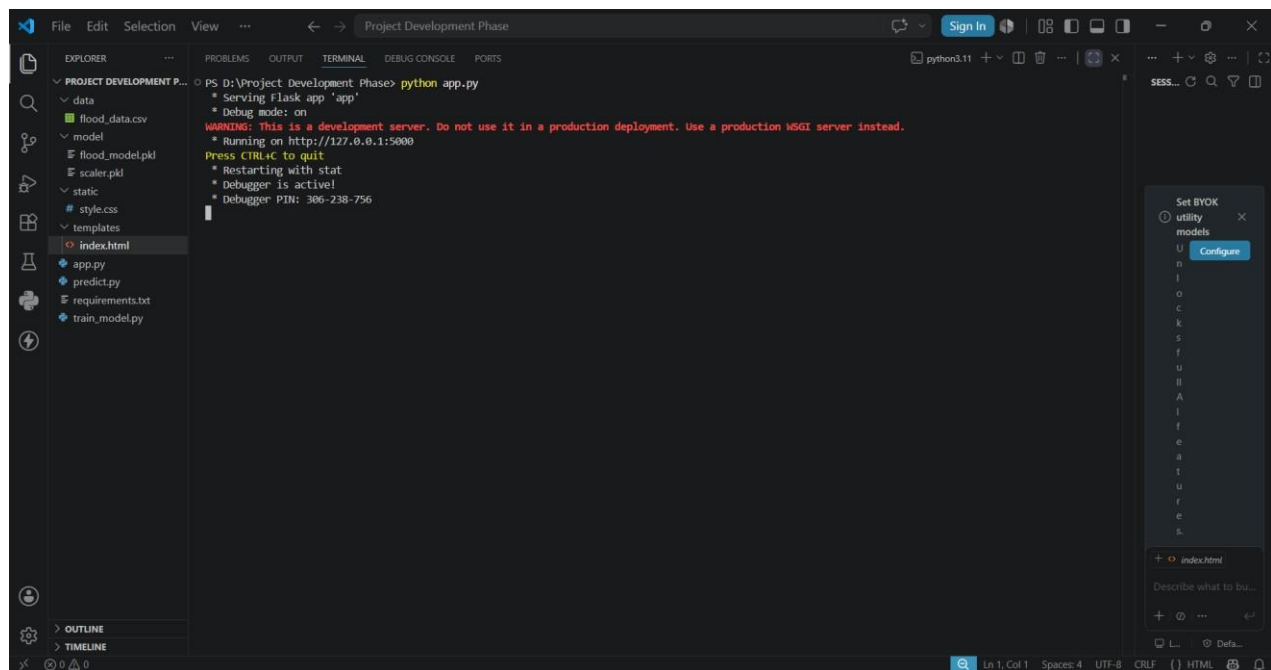


The screenshot shows the VS Code editor with the 'index.html' file open. The file contains HTML code for a web form to predict flood levels based on rainfall data. The form includes input fields for each month from July to December, each with a placeholder text indicating it is required. A 'Predict Flood' button is at the bottom of the form. Below the form, there is a conditional rendering block for the prediction result.

```

2 <html lang="en">
11 <body>
13 <div class="container">
19 <form action="/predict" method="POST">
37 <input type="number" step="any" name="jul" placeholder="July Rainfall" required>
38
39 <input type="number" step="any" name="aug" placeholder="August Rainfall" required>
40
41 <input type="number" step="any" name="sep" placeholder="September Rainfall" required>
42
43 <input type="number" step="any" name="oct" placeholder="October Rainfall" required>
44
45 <input type="number" step="any" name="nov" placeholder="November Rainfall" required>
46
47 <input type="number" step="any" name="dec" placeholder="December Rainfall" required>
48
49 <button type="submit">Predict Flood</button>
50
51 </form>
52
53 {% if prediction %}
54 <div class="result">
55 <h3>{{ prediction }}</h3>
56 </div>
57 {% endif %}
58
59 </div>
60
61 </body>
62 </html>
  
```

Screenshot-6 Flask server running



The screenshot shows the VS Code editor with the 'TERMINAL' panel open. The terminal displays the output of running the Flask application. It shows that the server is running on http://127.0.0.1:5000 and that the debugger is active. A warning message is also displayed, advising not to use this development server in production.

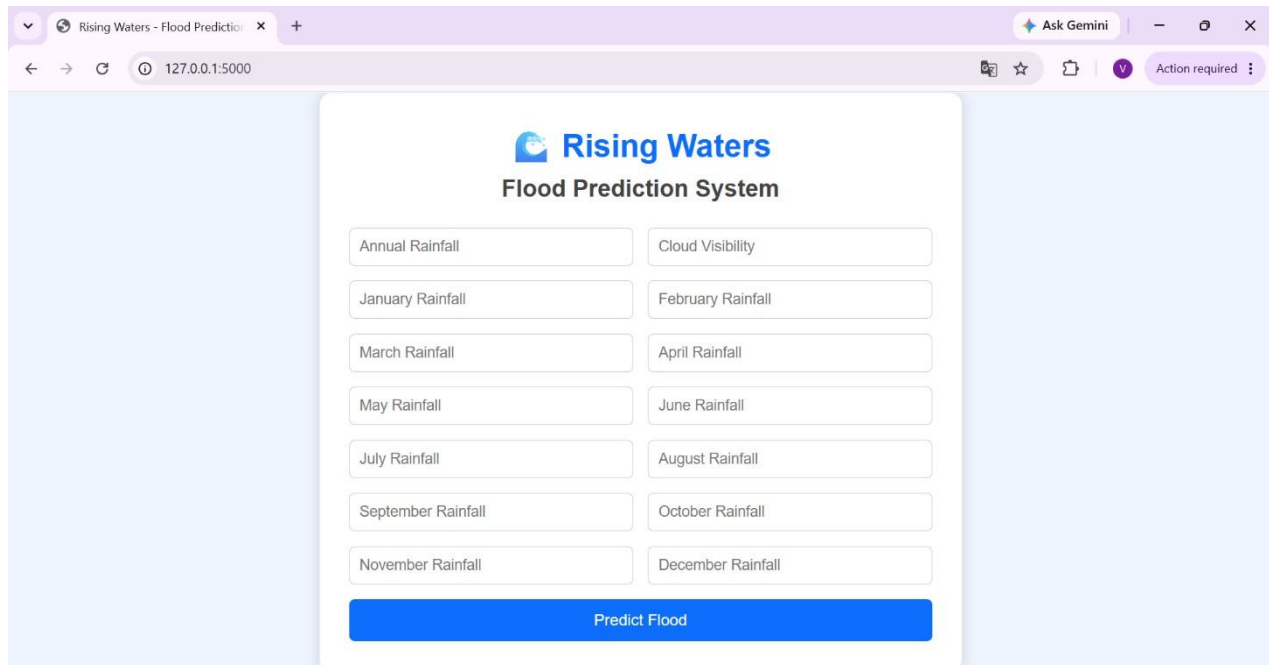
```

PS D:\Project Development Phase> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 306-238-756
  
```

Milestone 4: Prediction & Testing

The web application was tested using different flood-related input values. The trained model generated predictions indicating whether flooding was likely or not. Multiple test cases were executed to verify the correctness and reliability of the prediction system.

Screenshot-1 Home page of the application

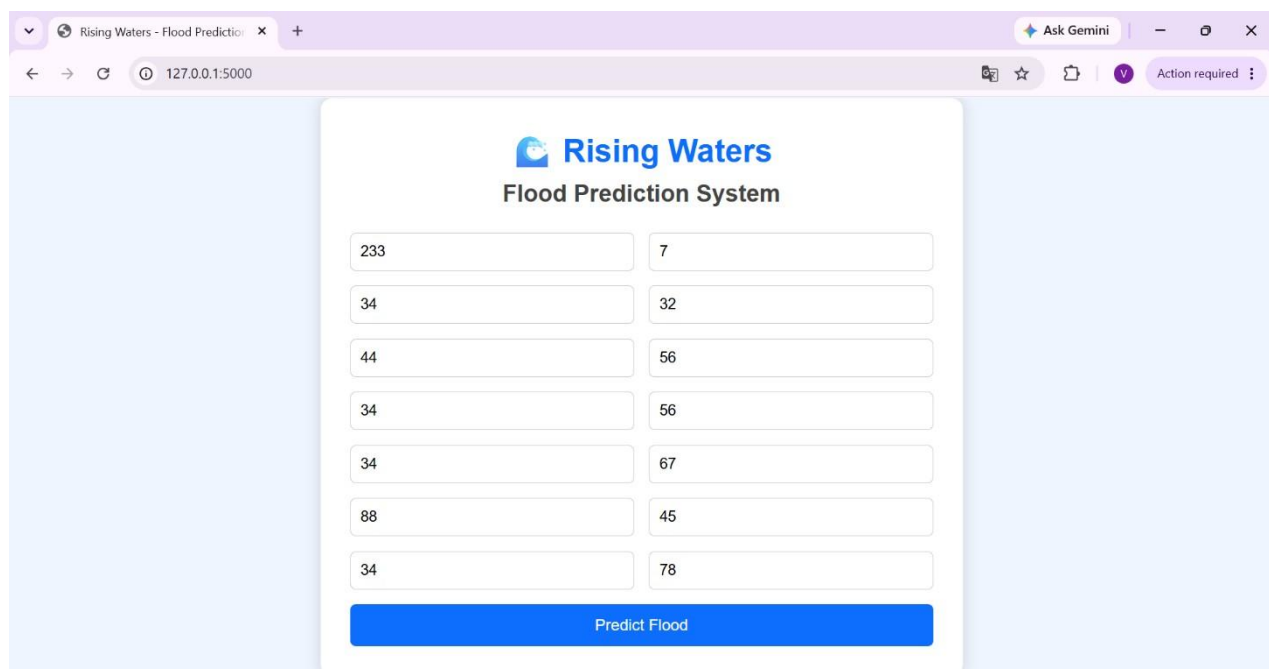


Rising Waters
Flood Prediction System

Annual Rainfall	Cloud Visibility
January Rainfall	February Rainfall
March Rainfall	April Rainfall
May Rainfall	June Rainfall
July Rainfall	August Rainfall
September Rainfall	October Rainfall
November Rainfall	December Rainfall

Predict Flood

Screenshot-2 Form filled with input values

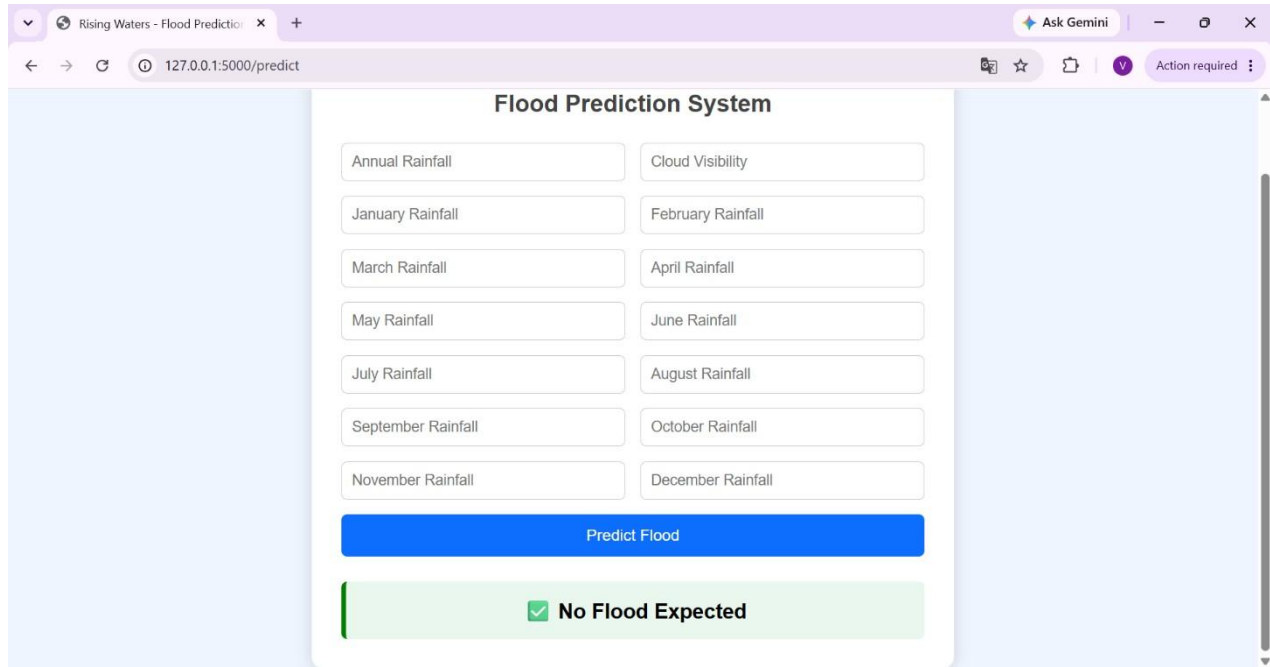


Rising Waters
Flood Prediction System

233	7
34	32
44	56
34	56
34	67
88	45
34	78

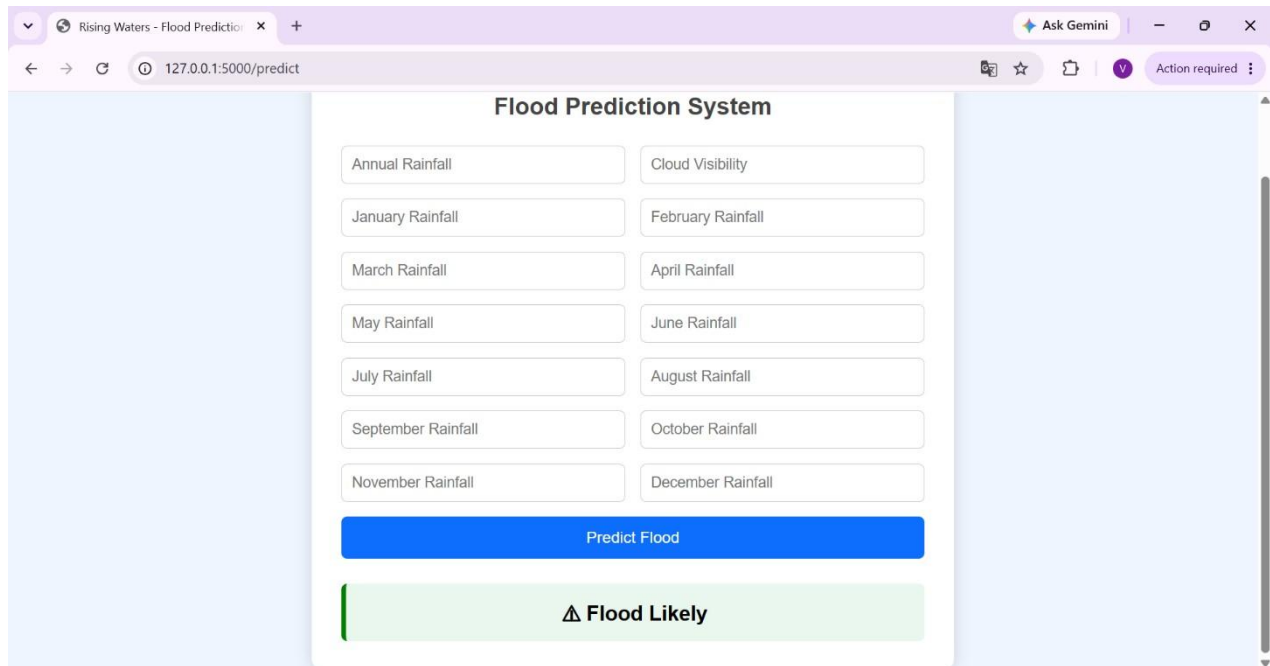
Predict Flood

Screenshot-3 Prediction showing no chance of floods



The screenshot shows a web browser window with the URL `127.0.0.1:5000/predict`. The page is titled "Flood Prediction System". It contains a form with 11 input fields for rainfall data: Annual Rainfall, Cloud Visibility, January Rainfall, February Rainfall, March Rainfall, April Rainfall, May Rainfall, June Rainfall, July Rainfall, August Rainfall, September Rainfall, October Rainfall, November Rainfall, and December Rainfall. Below the form is a blue button labeled "Predict Flood". At the bottom, a green banner displays a green checkmark icon and the text "No Flood Expected".

Screenshot-4 Prediction of floods occurrence



The screenshot shows the same "Flood Prediction System" web page. The form and "Predict Flood" button are identical to the previous screenshot. However, the bottom banner is green and displays a yellow warning triangle icon and the text "Flood Likely".

Milestone 5: Project Deployment & Documentation

The final milestone focused on organizing the project files, preparing documentation, testing the complete workflow, and uploading the project to GitHub. All source files, model files, templates, documentation, and project reports were verified before submission.

Screenshot-1 Git hub Repository


Rising-Waters
Public
Pin
Watch 0

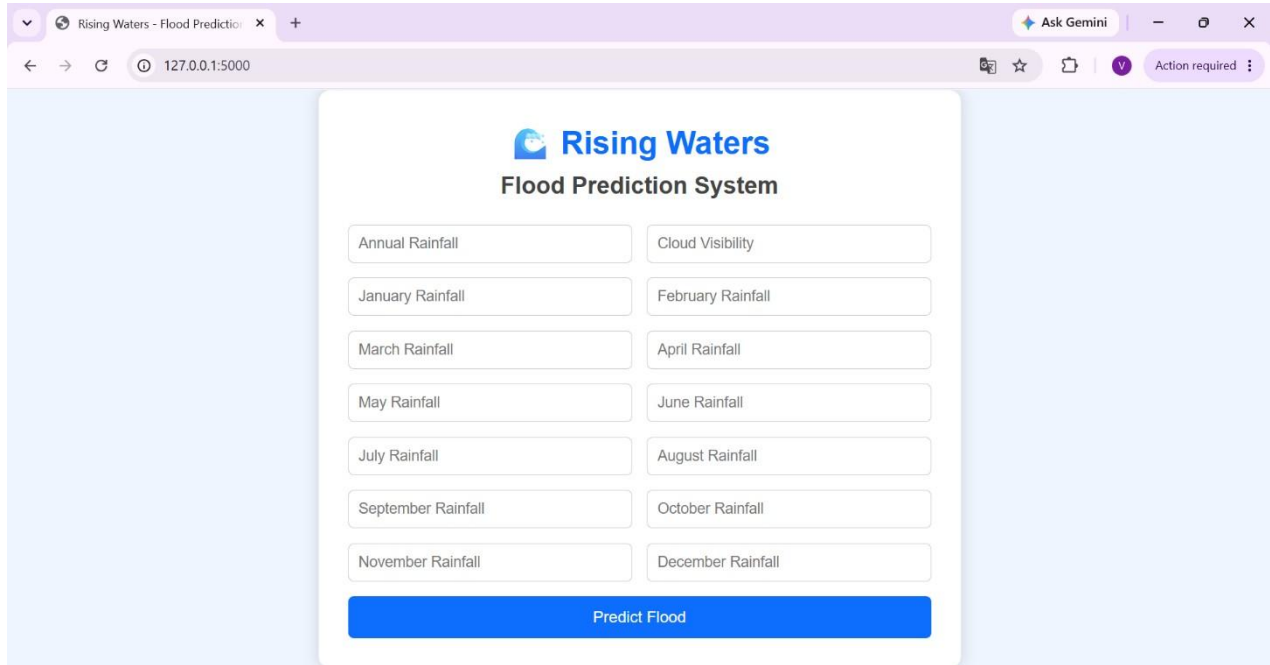
Main
1 Branch
0 Tags

+
Code

VineethaKuppala Create README.md		eb1d685 · 1 minute ago	2 Commits
data	initial project upload	52 minutes ago	
model	initial project upload	52 minutes ago	
static	initial project upload	52 minutes ago	
templates	initial project upload	52 minutes ago	
README.md	Create README.md	1 minute ago	
app.py	initial project upload	52 minutes ago	
predict.py	initial project upload	52 minutes ago	
requirements.txt	initial project upload	52 minutes ago	
train_model.py	initial project upload	52 minutes ago	

Exploring the website's web pages

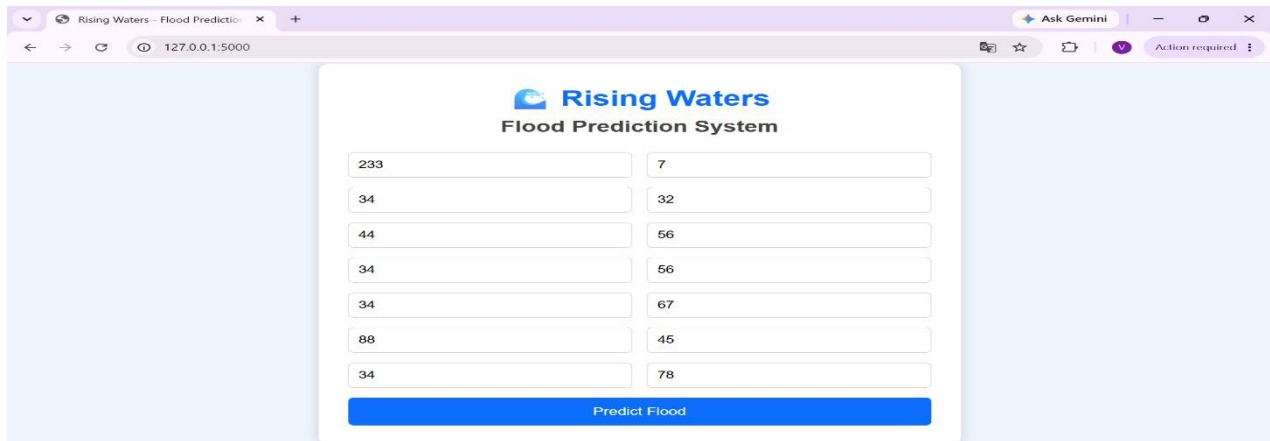
Home Page



The screenshot shows the home page of the 'Rising Waters Flood Prediction System'. The page has a light blue background. At the top, there's a header with the 'Rising Waters' logo and the title 'Flood Prediction System'. Below the header, there's a form with 12 input fields arranged in two columns. The left column contains: 'Annual Rainfall', 'January Rainfall', 'March Rainfall', 'May Rainfall', 'July Rainfall', 'September Rainfall', and 'November Rainfall'. The right column contains: 'Cloud Visibility', 'February Rainfall', 'April Rainfall', 'June Rainfall', 'August Rainfall', 'October Rainfall', and 'December Rainfall'. At the bottom of the form, there's a large blue button labeled 'Predict Flood'.

Description: The home page serves as the main interface of the Flood Prediction System. It provides users with a simple and user-friendly environment to access the prediction service. The page contains the project title, input form, and necessary instructions for entering environmental parameters required for flood prediction. The responsive design ensures ease of use across different devices and browsers.

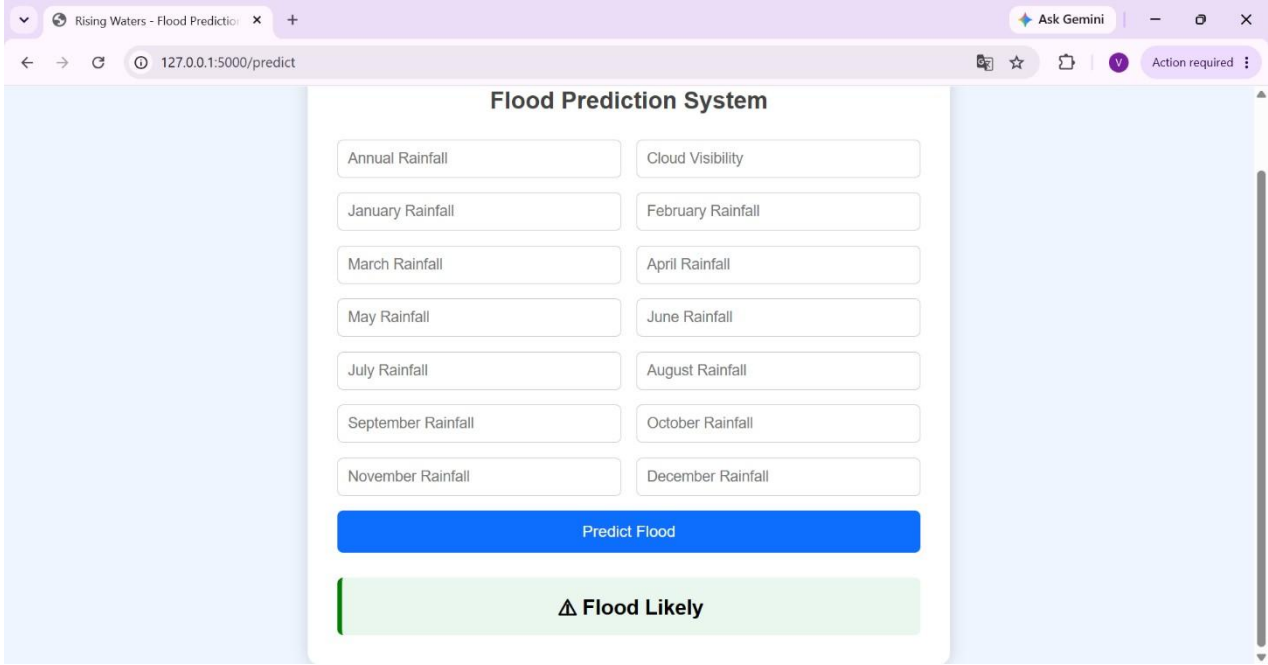
Page with input values



The screenshot shows the same 'Rising Waters Flood Prediction System' page, but with numerical values entered into the input fields. The values are: Annual Rainfall (233), January Rainfall (34), March Rainfall (44), May Rainfall (34), July Rainfall (34), September Rainfall (88), November Rainfall (34), Cloud Visibility (7), February Rainfall (32), April Rainfall (56), June Rainfall (56), August Rainfall (67), October Rainfall (46), and December Rainfall (78). The 'Predict Flood' button remains at the bottom.

Description: The input page allows users to enter the environmental and weather-related parameters required for flood prediction. The application validates the entered values before processing them. Once the user submits the form, the input data is pre-processed using the trained scaler and forwarded to the machine learning model for prediction.

Prediction Result



The screenshot shows a web browser window with the title "Rising Waters - Flood Prediction". The address bar shows the URL "127.0.0.1:5000/predict". The page content is titled "Flood Prediction System". It features a form with input fields for "Annual Rainfall", "Cloud Visibility", "January Rainfall", "February Rainfall", "March Rainfall", "April Rainfall", "May Rainfall", "June Rainfall", "July Rainfall", "August Rainfall", "September Rainfall", "October Rainfall", "November Rainfall", and "December Rainfall". Below the form is a blue button labeled "Predict Flood". At the bottom, a green box displays the prediction result: "⚠ Flood Likely".

Description: After receiving the user inputs, the trained XGBoost machine learning model analyzes the data and predicts the likelihood of flooding. The prediction result is displayed instantly on the web page, allowing users to understand the flood risk based on the provided environmental conditions. The system generates predictions quickly and accurately through the integrated Flask application.

Conclusion

The Flood Prediction Using Machine Learning project successfully demonstrates the integration of machine learning with a Flask-based web application to provide real-time flood risk predictions. The system preprocesses user inputs, applies a trained XGBoost model, and displays prediction results through a simple and interactive interface. The project showcases practical applications of data science, web development, and machine learning in addressing real-world environmental challenges. Future improvements can include integration with real-time weather APIs, cloud deployment, enhanced datasets, and interactive data visualization to further improve the system's accuracy and usability.